

PODSTAWY PROGRAMOWANIA – LABORATORIUM

Ćwiczenie 1

Aplikacja konsolowa, typy danych, zmienne i instrukcje sterujące

Przedmiot	Podstawy programowania (I1_12) – laboratorium
Kierunek / poziom	Informatyka, studia pierwszego stopnia, profil praktyczny, I rok, I semestr
Zajęcia	zajęcia 1–4
Czas trwania	4 godziny lekcyjne (4 × 45 min = 180 min)
Język i środowisko	C# 12, platforma .NET 8 (LTS), Microsoft Visual Studio 2022 (Community) lub VS Code + C# Dev Kit
Treści programowe	Lab. 1: tworzenie prostej aplikacji z interfejsem CLI; Lab. 2: stałe i zmienne, typy złożone (enum, struct, tuple); Lab. 4: instrukcje warunkowe i pętle
Literatura do zajęć	M. Lis, „C#. Praktyczny kurs”, Helion – lekcje 1–11 (rozdz. 1 i 2); M. Price, „C# 12 and .NET 8 – Modern Cross-Platform Development Fundamentals”, rozdz. 1–3

Jak pracować z tą instrukcją

- Każda część zaczyna się od **teorii praktycznej** z kompletnymi, przetestowanymi listingami – przepisuj je i uruchom, zanim przejdziesz do poleceń.
- Polecenia mają trzy poziomy: **podstawowy** (obowiązkowy dla wszystkich), **rozszerzony** (jedna gwiazdka) i **zaawansowany** (dwie gwiazdki) – dla osób, które skończyły wcześniej lub chcą ćwiczyć w domu.
- Polecenia oznaczone jako **punkt kontrolny** zgłaszasz prowadzącemu – na ich podstawie oceniana jest bieżąca praca na zajęciach.
- Każdy program zapisuj w osobnym projekcie lub osobnej metodzie (np. `Polecenie05()`), aby można było go pokazać w dowolnym momencie.

Cele i organizacja zajęć

Po zakończeniu ćwiczenia student potrafi samodzielnie utworzyć, skompilować, uruchomić i debugować aplikację konsolową w języku C#, dobrać odpowiedni typ danych do przechowywanej informacji, wczytać i zweryfikować dane od użytkownika oraz sterować przebiegiem programu za pomocą instrukcji warunkowych i pętli. Ćwiczenie łączy i porządkuje materiał dotychczasowych instrukcji 1 i 2, uzupełniając go o nowoczesne elementy języka (interpolacja łańcuchów, `TryParse`, wyrażenie `switch`, krotki).

Efekty uczenia się realizowane na zajęciach

- zna pojęcia: stała, zmienna, typ prosty, typ złożony, blok kodu, blok warunku, blok pętli (efekt 01 – II_W04),
- stosuje standardy zapisu bloków kodu i konwencje nazewnictwa języka C# (efekt 02 – II_W05),
- tworzy proste aplikacje z interfejsem tekstowym (CLI) i analizuje ich działanie w trybie krokowym (efekty 03 i 05 – II_U04, II_U18).

Harmonogram (4 × 45 min)

Godzina	Zakres	Polecenia
1 (0–45 min)	Środowisko .NET, struktura programu, kompilacja i uruchomienie, wyjście na konsolę, debugowanie	1–4
2 (45–90 min)	Typy danych, zmienne i stałe, wczytywanie danych, konwersje, operatory, klasa <code>Math</code>	5–9
3 (90–135 min)	Typy złożone: <code>enum</code> , <code>struct</code> , <code>tuple</code> ; instrukcje warunkowe <code>if/else</code> , <code>switch</code> , wyrażenie <code>switch</code> , operator <code>?:</code>	10–14
4 (135–180 min)	Pętle <code>for</code> , <code>while</code> , <code>do...while</code> , <code>foreach</code> ; <code>break</code> i <code>continue</code> ; pętle zagnieżdżone; menu programu konsolowego	15–21

Przygotowanie stanowiska

Na komputerze powinien być zainstalowany Visual Studio 2022 z pakietem roboczym „Programowanie aplikacji klasycznych dla platformy .NET” albo .NET 8 SDK i Visual Studio Code z rozszerzeniem C# Dev Kit. Instrukcje z poprzednich lat odnosiły się do Visual Studio 2010 – wszystkie przedstawione tam programy działają również w .NET 8, zmienił się jedynie wygląd kreatora projektu.

Część 1. Środowisko i pierwszy program

1.1. Od kodu źródłowego do działającego programu

Program w języku C# nie jest tłumaczony bezpośrednio na kod maszynowy procesora. Kompilator (Roslyn) zamienia kod źródłowy (pliki `.cs`) na **kod pośredni IL** (Intermediate Language), zapisywany w pliku `.dll`. Podczas uruchomienia środowisko uruchomieniowe **CLR** (Common Language Runtime) kompiluje IL „w locie” (kompilator **JIT** – Just-In-Time) do instrukcji konkretnego procesora. Dzięki temu ten sam program działa w systemach Windows, Linux i macOS, a CLR dodatkowo zarządza pamięcią (moduł odśmiecania pamięci – **GC**, Garbage Collector) i pilnuje bezpieczeństwa typów.

Etap	Narzędzie	Wynik
Edycja	Visual Studio / VS Code	pliki <code>*.cs</code> , plik projektu <code>*.csproj</code>
Kompilacja (Build, Ctrl+Shift+B)	kompilator C# (Roslyn)	<code>bin\Debug\net8.0\Nazwa.dll + Nazwa.exe</code> (program uruchamiający)
Uruchomienie (F5 / Ctrl+F5)	CLR + kompilator JIT	kod maszynowy wykonywany przez procesor

1.2. Tworzenie projektu

W Visual Studio 2022: **Plik** → **Nowy** → **Projekt** → **Aplikacja konsolowa (C#)**. Podaj nazwę (np. `Lab01`), wybierz platformę **.NET 8.0 (obsługa długoterminowa)** i zaznacz opcję **„Nie używaj instrukcji najwyższego poziomu”** (ang. Do not use top-level statements) – otrzymasz wtedy klasyczną strukturę programu z jawną metodą `Main`, taką jak w książce M. Lisa i w dotychczasowych instrukcjach.

Ten sam projekt można utworzyć z wiersza poleceń (działa także w Linuksie):

```
dotnet new console -n Lab01 --use-program-main
cd Lab01
dotnet run
```

1.3. Struktura programu

Listing 1.1. Minimalny program konsolowy z komentarzami

```
using System;                                // import przestrzeni nazw System (Console, Math, ...)

namespace Lab01                               // własna przestrzeń nazw – grupuje klasy projektu
{
    class Program                             // klasa zawierająca punkt wejścia programu
    {
        static void Main(string[] args)      // metoda Main – od niej zaczyna się wykonanie
        {
            Console.WriteLine("Witaj na laboratorium z Podstaw programowania!");
            Console.WriteLine("Naciśnij dowolny klawisz, aby zakończyć...");
            Console.ReadKey(true);           // zatrzymanie programu do naciśnięcia klawisza
        }
    }
}
```

- `using System;` – pozwala pisać `Console` zamiast pełnej nazwy `System.Console`.
- `namespace` – przestrzeń nazw grupująca klasy; zapobiega konfliktom nazw w większych projektach.
- `class Program` – w C# każdy kod wykonywalny musi znajdować się wewnątrz klasy.

- `static void Main(string[] args)` – punkt wejścia; `static` oznacza, że metodę można wywołać bez tworzenia obiektu, `void` – że nie zwraca wartości, `args` – argumenty wiersza poleceń.
- Nawiasy klamrowe `{ }` wyznaczają **blok kodu**; każda instrukcja kończy się średnikiem `;`.
- C# rozróżnia wielkość liter: `Main`, `main` i `MAIN` to trzy różne nazwy.

Listing 1.2. Ten sam program w stylu instrukcji najwyższego poziomu (dla porównania)

```
// Program.cs w szablonie .NET 6+ (instrukcje najwyższego poziomu – top-level statements)
Console.WriteLine("Witaj!");
// Kompilator sam „dopisuje” klasę Program i metodę Main.
// Na naszych zajęciach stosujemy jawną metodę Main – ułatwia to naukę funkcji i klas.
```

Konwencje nazewnictwa w C# (warto stosować od pierwszych zajęć)

- klasy, metody, właściwości, stałe: **PascalCase** – `Program`, `ObliczPole`, `MaxLiczbaProb`;
- zmienne lokalne i parametry: **camelCase** – `liczbaStudentow`, `sredniaOcen`;
- nazwy powinny mówić „co” przechowują: `dlugoscBoku` zamiast `x1`; w identyfikatorach unikamy polskich znaków;
- jedno polecenie w linii, wcięcie 4 spacje na każdy poziom zagnieżdżenia (Ctrl+K, Ctrl+D formatuje dokument).

1.4. Wyprowadzanie danych na ekran

Metoda `Console.Write` wypisuje tekst i pozostawia kursor w tej samej linii, `Console.WriteLine` dodatkowo przechodzi do nowej linii. Do budowania komunikatów z wartościami zmiennych zalecana jest **interpolacja łańcuchów** – znak `$` przed cudzysłowem i wyrażenia w nawiasach klamrowych. Po dwukropku można podać **format**, a po przecinku **szerokość pola**.

Listing 1.3. Sposoby wypisywania i formatowania danych (fragment – umieść wewnątrz metody Main)

```
string imie = "Anna";
int wiek = 20;
double srednia = 4.356;

Console.Write("Tekst bez przejścia do nowej linii. ");
Console.WriteLine("A ten tekst kończy linię.");

// 1) łączenie łańcuchów operatorem +
Console.WriteLine("Imię: " + imie + ", wiek: " + wiek);
// 2) formatowanie złożone – numerowane miejsca {0}, {1}, ...
Console.WriteLine("Imię: {0}, wiek: {1}", imie, wiek);
// 3) interpolacja łańcuchów (zalecana) – znak $ przed cudzysłowem
Console.WriteLine($"Imię: {imie}, wiek: {wiek}, średnia: {srednia:F2}");

// formatowanie liczb i wyrównanie w kolumnach
Console.WriteLine($"|{wiek,6}|{srednia,10:F3}|{imie,-8}|"); // ,6 = do prawej, ,-8 = do lewej
Console.WriteLine($"|{0.256:P1}|{1234567.891:N2}|{255:X}|{42:D5}|");

// sekwencje specjalne
Console.WriteLine("Kolumna1\tKolumna2\nNowa linia, cudzysłów: \" i ukośnik: \\\");
Console.WriteLine(@"C:\Users\student\Pulpit"); // łańcuch dosłowny (verbatim)
```

Format	Znaczenie	Przykład	Wynik (kultura pl-PL)
F2	stałoprzecinkowy, 2 miejsca po przecinku	{3.14159:F2}	3,14

Format	Znaczenie	Przykład	Wynik (kultura pl-PL)
N2	z separatorem tysięcy	{1234567.891:N2}	1 234 567,89
P1	procentowy	{0.256:P1}	25,6%
D5	całkowity, dopełniony zerami	{42:D5}	00042
X	szesnastkowy	{255:X}	FF
C	walutowy	{19.9:C}	19,90 zł
,8 / , -8	pole o szerokości 8, do prawej / do lewej	{x,8}	42

1.5. Debugowanie – praca krokowa

Debugger pozwala zatrzymać program w wybranym miejscu i obserwować wartości zmiennych. Umiejętność pracy krokowej jest kluczowa – większość błędów logicznych znajduje się szybciej debugerem niż przez „wpatrywanie się” w kod.

Klawisz	Działanie
F9	ustawia/usuwa punkt przerwania (breakpoint) w bieżącej linii (czerwona kropka)
F5	uruchamia program z debugerem (zatrzyma się na punkcie przerwania)
Ctrl+F5	uruchamia bez debugera – okno konsoli nie zamyka się automatycznie
F10	krok nad (Step Over) – wykonuje linię, nie wchodząc do wywoływanych metod
F11	krok do (Step Into) – wchodzi do wnętrza wywoływanej metody
Shift+F11	krok wyjścia (Step Out) – kończy bieżącą metodę i wraca do miejsca wywołania
okna Locals / Watch / Autos	podgląd i modyfikacja wartości zmiennych w trakcie zatrzymania

Najczęstsze komunikaty kompilatora

Listing 1.4. Kod celowo zawierający typowe błędy

```
int x = 5; // CS1002: oczekiwano ;
Console.WriteLine(y); // CS0103: nazwa 'y' nie istnieje w bieżącym kontekście
int z = "10"; // CS0029: nie można niejawnie przekonwertować string na int
int w;
Console.WriteLine(w); // CS0165: użycie nieprzypisanej zmiennej lokalnej 'w'
if (x > 3); // ostrzeżenie CS0642: pusta instrukcja – średnik kończy if!
{
    Console.WriteLine("Ten blok wykona się ZAWSZE");
}
```

Okno **Lista błędów** (Widok → Lista błędów, Ctrl+\, E) pokazuje kod błędu (np. **CS0103**), opis i numer linii. Kod błędu warto wpisać w wyszukiwarkę – dokumentacja Microsoft Learn zawiera opis i przykłady poprawek dla każdego komunikatu.

Polecenia – część 1

Polecenie 1. Pierwszy projekt i jego pliki

poziom podstawowy · orientacyjny czas: 8 min

Utwórz projekt aplikacji konsolowej **Lab01** (.NET 8, bez instrukcji najwyższego poziomu). Uruchom go klawiszem F5, a następnie Ctrl+F5. Czym różni się zachowanie okna konsoli?

Odszukaj na dysku katalog projektu (prawy przycisk na projekcie → „Otwórz folder w Eksploratorze plików”). Wskaż plik **.csproj**, plik **Program.cs** oraz pliki wynikowe w katalogu **bin\Debug\net8.0**. Uruchom plik **.exe** bezpośrednio z Eksploratora.

Polecenie 2. Wizytówka w konsoli

poziom podstawowy · orientacyjny czas: 10 min

Napisz program wyświetlający „wizytówkę” w ramce z gwiazdek: imię i nazwisko, kierunek studiów, numer albumu i adres e-mail. Każda linia ramki ma mieć tę samą długość – użyj wyrównania w polu o stałej szerokości (np. `{tekst, -30}`).

Zmień kolor napisów nagłówka za pomocą `Console.ForegroundColor = ConsoleColor.Cyan;` i przywróć domyślny przez `Console.ResetColor();`.

Przykładowy przebieg programu:

```
*****
* Jan Kowalski                *
* Informatyka, I rok          *
* nr albumu: 12345           *
* jan.kowalski@student.edu.pl *
*****
```

Polecenie 3. Formatowanie liczb

poziom podstawowy · orientacyjny czas: 8 min

Zadeklaruj zmienne: `double cena = 1234.5678; double rabat = 0.075; int ilosc = 7;`

Wyświetl: cenę z dwoma miejscami po przecinku, cenę jako kwotę w złotych (format C), rabat w procentach, ilość dopełnioną zerami do 4 cyfr oraz wartość zamówienia `cena * ilosc * (1 - rabat)` w polu o szerokości 15 znaków wyrównaną do prawej.

Polecenie 4. Śledzenie programu w debuggerze

poziom podstawowy · orientacyjny czas: 15 min

Przepisz do metody **Main** fragment z listingu 1.9 (operatory – część 2 instrukcji). Ustaw punkt przerwania w pierwszej linii i wykonaj program krokowo (F10). Po każdym kroku zapisz w zeszycie wartości zmiennych **licznik**, **n**, **p1**, **p2** widoczne w oknie Locals.

Wyjaśnij różnicę między **n++** i **++n** na podstawie zaobserwowanych wartości.

Następnie wklej do programu kod z listingu 1.4 (błędy). Odczytaj komunikaty z listy błędów i popraw kolejno wszystkie błędy.

Punkt kontrolny – zgłoś prowadzącemu zakończenie realizacji tego polecenia.

Część 2. Typy danych, zmienne, stałe i konwersje

2.1. Typy proste

C# jest językiem **silnie typowanym**: każda zmienna ma określony typ, którego nie można zmienić, a kompilator nie pozwoli np. przypisać tekstu do zmiennej liczbowej. Typ decyduje o zakresie wartości, zajmowanej pamięci i dozwolonych operacjach. Słowa kluczowe typów (`int`, `double`...) są aliasami struktur platformy .NET (`Int32`, `Double`...) – dotychczasowe instrukcje używały nazw .NET, oba zapisy są równoważne.

Słowo kluczowe	Typ .NET	Rozmiar	Zakres / uwagi	Przykład literału
<code>byte</code>	Byte	1 B	0 ... 255	<code>255</code>
<code>short</code>	Int16	2 B	-32 768 ... 32 767	<code>-1200</code>
<code>int</code>	Int32	4 B	ok. $\pm 2,1$ mld – domyślny typ całkowity	<code>42</code>
<code>long</code>	Int64	8 B	ok. $\pm 9,2 \cdot 10^{18}$	<code>5_000_000_000L</code>
<code>float</code>	Single	4 B	~7 cyfr znaczących	<code>3.14f</code>
<code>double</code>	Double	8 B	~15–16 cyfr znaczących – domyślny rzeczywisty	<code>2.5, 1e-9</code>
<code>decimal</code>	Decimal	16 B	28–29 cyfr, arytmetyka dziesiętna – kwoty pieniężne	<code>19.99m</code>
<code>bool</code>	Boolean	1 B	<code>true / false</code>	<code>true</code>
<code>char</code>	Char	2 B	jeden znak Unicode (UTF-16)	<code>'A', '\n'</code>
<code>string</code>	String	zmienny	łańcuch znaków (typ referencyjny, niezmienny)	<code>"Radom"</code>

W kodzie źródłowym separatorem części ułamkowej jest zawsze **kropka** (`2.5`), natomiast przy wyświetlaniu i wczytywaniu liczb .NET stosuje ustawienia regionalne systemu – w polskim Windows jest to **przecinek** (`2,5`). Znak podkreślenia w literałach (`1_000_000`) poprawia czytelność i jest ignorowany przez kompilator.

2.2. Deklaracja, inicjalizacja, stałe i var

- Deklaracja: `int liczba;` – zmienna lokalna **musi** zostać zainicjalizowana przed pierwszym odczytem (błąd CS0165).
- Deklaracja z inicjalizacją: `double srednia = 0.0;`, kilka zmiennych jednego typu: `int a = 1, b = 2;`.
- Stała: `const double Vat = 0.23;` – wartość ustalona w czasie kompilacji, próba zmiany to błąd kompilacji. Stałe eliminują „magiczne liczby” z kodu.
- `var` – kompilator ustala typ na podstawie wyrażenia po prawej stronie (`var x = 2.5;` → `double`). Typ nadal jest statyczny; `var` stosujemy, gdy typ jest oczywisty.
- Zasięg zmiennej kończy się wraz z blokiem `{ }`, w którym ją zadeklarowano.

Listing 1.5. Dzielenie całkowite, precyzja liczb rzeczywistych, przepelnienie, stałe (fragment – umieść wewnątrz metody Main)

```
int a = 7, b = 2;
Console.WriteLine(a / b);           // 3    - dzielenie całkowite (część ułamkowa jest odcinana)
Console.WriteLine(a % b);           // 1    - reszta z dzielenia (modulo)
Console.WriteLine(a / 2.0);         // 3,5  - jeden argument rzeczywisty => wynik rzeczywisty
Console.WriteLine((double)a / b);   // 3,5  - rzutowanie jawne przed dzieleniem
```

```
double x = 0.1 + 0.2;
Console.WriteLine(x == 0.3);           // False! - błąd reprezentacji binarnej
Console.WriteLine(Math.Abs(x - 0.3) < 1e-9); // True - porównanie z tolerancją
decimal m = 0.1m + 0.2m;
Console.WriteLine(m == 0.3m);          // True - decimal liczy dziesiętnie (finanse)

int duza = int.MaxValue;
duza = duza + 1;                       // przepełnienie - „zawinięcie” do int.MinValue
Console.WriteLine(duza);                // -2147483648
long bezpieczna = (long)int.MaxValue + 1;
Console.WriteLine(bezpieczna);          // 2147483648

const double G = 9.81;                 // stała - wartości nie można zmienić
var t = 2.5;                           // var - typ wywnioskowany przez kompilator (double)
Console.WriteLine($"Droga spadku po {t} s: {G * t * t / 2:F2} m");
```

Trzy pułapki, które warto zapamiętać

- **Dzielenie dwóch liczb całkowitych daje wynik całkowity:** $7 / 2$ to 3 . Jeśli potrzebny jest wynik rzeczywisty, przynajmniej jeden argument musi być typu `double`.
- **Liczb `double` nie porównujemy operatorem `==`** – $0,1$ nie ma dokładnej reprezentacji binarnej. Porównujemy różnicę z małą tolerancją (np. `1e-9`), a do pieniędzy używamy `decimal`.
- **Przepełnienie typu całkowitego nie jest domyślnie zgłaszane** – wartość „zawija się”. Blok `checked { ... }` lub opcja projektu „Sprawdzaj przepełnienie arytmetyczne” powoduje zgłoszenie wyjątku `OverflowException`.

2.3. Wczytywanie danych z klawiatury

`Console.ReadLine()` zwraca zawsze tekst (`string`) – wszystko, co wpisano do naciśnięcia Enter. Aby wykonać obliczenia, tekst trzeba przekonwertować na liczbę metodą `Parse` danego typu. Jeżeli użytkownik wpisze coś, czego nie da się przekonwertować, `Parse` zgłosi wyjątek `FormatException` i program zakończy się błędem.

Listing 1.6. Wczytywanie tekstu i liczb

```
using System;

class Program
{
    static void Main()
    {
        Console.Write("Podaj swoje imię: ");
        string imie = Console.ReadLine();           // ReadLine zawsze zwraca tekst (string)

        Console.Write("Podaj rok urodzenia: ");
        int rok = int.Parse(Console.ReadLine());    // konwersja tekstu na liczbę całkowitą

        Console.Write("Podaj wzrost w metrach (np. 1,82): ");
        double wzrost = double.Parse(Console.ReadLine());

        int wiek = DateTime.Now.Year - rok;
        Console.WriteLine($"{imie}, w tym roku kończysz {wiek} lat, wzrost: {wzrost:F2} m.");
    }
}
```


Bezpieczniejszą metodą jest `TryParse` – nie zgłasza wyjątku, lecz zwraca `true/false` informując, czy konwersja się udała, a wynik przekazuje przez parametr `out`. To podstawowy wzorec walidacji danych wejściowych, stosowany we wszystkich kolejnych ćwiczeniach.

Listing 1.7. Bezpieczna konwersja z `TryParse` (fragment – umieść wewnątrz metody `Main`)

```
Console.WriteLine("Podaj liczbę całkowitą: ");
string tekst = Console.ReadLine();

if (int.TryParse(tekst, out int liczba))
{
    Console.WriteLine($"Poprawnie wczytano: {liczba}, jej kwadrat to {liczba * liczba}");
}
else
{
    Console.WriteLine($"\"{tekst}\" nie jest poprawną liczbą całkowitą.");
}
```

Ostrzeżenia CS8600/CS8604 i polskie znaki

W projektach .NET 8 domyślnie włączona jest analiza wartości `null` (`<Nullable>enable</Nullable>` w pliku `.csproj`). Ponieważ `ReadLine()` teoretycznie może zwrócić `null`, kompilator pokazuje ostrzeżenia. Na tym etapie można je zignorować, zapisać `string? tekst = Console.ReadLine();` albo wyłączyć opcję, zmieniając w pliku projektu `enable` na `disable`.

Jeżeli w konsoli Windows polskie znaki wyświetlają się niepoprawnie, dodaj na początku `Main`:

`Console.OutputEncoding = System.Text.Encoding.UTF8;` (oraz analogicznie `Console.InputEncoding`).

2.4. Konwersje typów

Konwersja **niejawna** zachodzi automatycznie, gdy nie grozi utratą informacji (np. `int` → `long` → `double`). Konwersja **jawna** (rzutowanie) wymaga zapisu typu w nawiasie: `(int)3.99` – część ułamkowa jest obcinana. Konwersje między tekstem a liczbą wykonują metody `Parse/TryParse`, `ToString()` oraz klasa `Convert`.

Listing 1.8. Konwersje jawne i niejawne oraz funkcje klasy `Math` (fragment – umieść wewnątrz metody `Main`)

```
int i = 65;
double d = i; // niejawna (bezpieczna) konwersja int -> double
int j = (int)3.99; // jawne rzutowanie - obcięcie, wynik 3
int k = (int)Math.Round(3.5); // zaokrąglenie „bankowe” - wynik 4 (do parzystej)
int k2 = (int)Math.Round(2.5); // wynik 2 (!)
char c = (char)i; // 'A' - kod Unicode 65
int kod = 'a'; // 97
string s = i.ToString(); // liczba -> tekst
int z = Convert.ToInt32("123");
Console.WriteLine($"{d} {j} {k} {k2} {c} {kod} {s} {z}");
Console.WriteLine($"{Math.Sqrt(2):F4} {Math.Pow(2, 10)} {Math.PI:F5} {Math.Max(3, 8)}");
```

Metoda klasy <code>Math</code>	Opis	Przykład → wynik
<code>Math.Abs(x)</code>	wartość bezwzględna	<code>Math.Abs(-4.5)</code> → 4,5
<code>Math.Sqrt(x)</code>	pierwiastek kwadratowy	<code>Math.Sqrt(16)</code> → 4
<code>Math.Pow(x, y)</code>	potęga x^y (wynik <code>double</code>)	<code>Math.Pow(2, 10)</code> → 1024
<code>Math.Round(x, n)</code>	zaokrąglenie do <code>n</code> miejsc (domyślnie „bankowe”)	<code>Math.Round(2.345, 2)</code> → 2,34 lub 2,35*

Metoda klasy Math	Opis	Przykład → wynik
Math.Floor / Ceiling	zaokrąglenie w dół / w górę	Math.Ceiling(2.1) → 3
Math.Min / Max	mniejsza / większa z dwóch wartości	Math.Max(3, 8) → 8
Math.Sin, Cos, Tan	funkcje trygonometryczne (kąt w radianach)	Math.Sin(Math.PI / 2) → 1
Math.PI, Math.E	stałe matematyczne	3,14159..., 2,71828...

* Liczba 2,345 nie ma dokładnej reprezentacji w `double`, dlatego wynik zaokrąglenia może zaskakiwać. Do zaokrągleń „szkolnych” używaj `Math.Round(x, 2, MidpointRounding.AwayFromZero)`.

2.5. Operatory

Listing 1.9. Operatory arytmetyczne, inkrementacji i logiczne (fragment – umieść wewnątrz metody Main)

```
int licznik = 5;
licznik++;           // 6 (inkrementacja)
licznik += 10;       // 16 (to samo co licznik = licznik + 10)
licznik %= 7;        // 2

int n = 5;
int p1 = n++;        // p1 = 5, potem n = 6 (postinkrementacja)
int p2 = ++n;        // n = 7, potem p2 = 7 (preinkrementacja)

int wiek = 19;
bool student = true;
bool znizka = (wiek < 26 && student) || wiek >= 65; // && - „i”, || - „lub”
bool nieparzysta = !(n % 2 == 0);                 // ! - negacja
Console.WriteLine($"{licznik} {p1} {p2} {znizka} {nieparzysta}");
```

Grupa (od najwyższego priorytetu)	Operatory
jednoargumentowe	<code>++</code> <code>--</code> <code>!</code> <code>-</code> (typ)
multiplikatywne	<code>*</code> <code>/</code> <code>%</code>
addytywne	<code>+</code> <code>-</code>
relacyjne	<code><</code> <code>></code> <code><=</code> <code>>=</code>
równości	<code>==</code> <code>!=</code>
logiczne	<code>&&</code> (i, skrócone obliczanie), <code> </code> (lub, skrócone obliczanie)
warunkowy	<code>?:</code>
przypisania	<code>=</code> <code>+=</code> <code>-=</code> <code>*=</code> <code>/=</code> <code>%=</code>

Operatory `&&` i `||` obliczają drugi argument tylko wtedy, gdy jest to potrzebne (np. w `x != 0 && 10 / x > 1` dzielenie nie wykona się dla `x == 0`). Operatory `&` i `|` użyte w instrukcji z poprzednich lat zawsze obliczają oba argumenty – w warunkach należy stosować wersje podwójne. W razie wątpliwości co do priorytetu zawsze można dodać nawiasy – poprawiają też czytelność.

Polecenia – część 2

Polecenie 5. Pole i obwód prostokąta

poziom podstawowy · orientacyjny czas: 8 min

Napisz program, który wczytuje długości boków prostokąta `a` i `b` (liczby rzeczywiste), a następnie wyświetla pole, obwód i długość przekątnej z dokładnością do 3 miejsc po przecinku. Komunikaty powinny informować użytkownika, co ma wpisać.

Przetestuj program dla danych z przecinkiem (2,5) i z kropką (2.5). Co się dzieje i dlaczego?

Polecenie 6. Kalkulator BMI z walidacją

poziom podstawowy · orientacyjny czas: 10 min

Program wczytuje masę ciała [kg] i wzrost [cm], oblicza wskaźnik BMI = masa / wzrost² (wzrost w metrach!) i wyświetla wynik z jednym miejscem po przecinku.

Do wczytywania użyj `double.TryParse`. Jeżeli którakolwiek wartość jest niepoprawna lub niedodatnia, program wyświetla komunikat o błędzie i kończy działanie (instrukcja `return`; w `Main`).

Polecenie 7. Rozkład kwoty na banknoty i monety

poziom podstawowy · orientacyjny czas: 10 min

Wczytaj kwotę całkowitą w złotych (np. 1 788). Wykorzystując wyłącznie operatory `/` oraz `%` dla liczb całkowitych, wyznacz najmniejszą liczbę banknotów 500, 200, 100, 50, 20, 10 zł i monet 5, 2, 1 zł potrzebnych do jej wypłacenia. Wyświetl tylko niezerowe pozycje.

Przykładowy przebieg programu:

```
Kwota: 1788
500 zł x 3
200 zł x 1
50 zł x 1
20 zł x 1
10 zł x 1
5 zł x 1
2 zł x 1
1 zł x 1
```

Wskazówka: po wyznaczeniu liczby banknotów danego nominału `n = kwota / nominal` zmniejsz kwotę: `kwota %= nominal`. W tej części nie musisz jeszcze używać pętli ani tablic – do tego zadania wrócisz w ćwiczeniu 2.

Polecenie 8. Obliczanie wyrażenia i stałe

poziom podstawowy · orientacyjny czas: 10 min

Zdefiniuj stałą `const double G = 6.674e-11`; (stała grawitacji) i oblicz siłę przyciągania między Ziemią (5,972·10²⁴ kg) i Księżycem (7,348·10²² kg) oddległymi o 384 400 km ze wzoru $F = G \cdot m_1 \cdot m_2 / r^2$. Wynik wyświetl w notacji naukowej (format `E3`).

Oblicz i wyświetl z dokładnością do 4 miejsc wartość wyrażenia $(\sqrt{5} + 1) / 2$ (złoty podział). Sprawdź, czy wynik podniesiony do kwadratu jest równy wynikowi powiększonemu o 1 – porównaj wartości „z tolerancją”.

Polecenie 9. Przepętnienie i precyzja – eksperyment

poziom rozszerzony (*) · orientacyjny czas: 12 min

Oblicz silnię liczb od 10 do 25, kolejno dla zmiennej typu `int`, `long`, `double` i `decimal` (możesz po prostu mnożyć w kolejnych liniach lub użyć pętli `for`, jeśli już ją znasz). Od której wartości n wyniki typu `int` i `long` stają się błędne? Umieść obliczenia w bloku `checked { }` i zaobserwuj zachowanie programu.

Zsumuj dziesięć razy wartość 0,1 w zmiennej `double` i w zmiennej `decimal`. Wyświetl wynik z 17 miejscami po przecinku (F17) i porównaj z wartością 1.

Punkt kontrolny – zgłoś prowadzącemu zakończenie realizacji tego polecenia.

Część 3. Typy złożone i instrukcje warunkowe

3.1. Typ wyliczeniowy enum

Typ wyliczeniowy definiuje **zbiór nazwanych stałych całkowitych**. Zamiast zapamiętywać, że 3 oznacza środę, piszemy `DzienTygodnia.Sroda` – kod jest czytelniejszy, a kompilator nie pozwoli przypisać przypadkowej wartości innego typu. Domyślnie pierwsza stała ma wartość 0, a kolejne są numerowane co 1; wartości można też nadać jawnie. Definicję `enum` umieszczamy poza metodą (w przestrzeni nazw lub w klasie).

Listing 1.10. Definicja i użycie typu wyliczeniowego

```
using System;

enum DzienTygodnia { Poniedzialek = 1, Wtorek, Sroda, Czwartek, Piatek, Sobota, Niedziela }

enum Ocena { Niedostateczna = 2, Dostateczna = 3, Dobra = 4, BardzoDobra = 5 }

class Program
{
    static void Main()
    {
        DzienTygodnia dzien = DzienTygodnia.Sroda;
        Console.WriteLine($"Dzień: {dzien}, numer: {(int)dzien}");    // Sroda, 3

        int numer = 6;
        // rzutowanie liczby na enum
        DzienTygodnia d2 = (DzienTygodnia)numer;
        bool weekend = d2 == DzienTygodnia.Sobota || d2 == DzienTygodnia.Niedziela;
        Console.WriteLine($"{d2} - weekend: {weekend}");

        // wypisanie wszystkich wartości typu wyliczeniowego
        foreach (Ocena o in Enum.GetValues(typeof(Ocena)))
            Console.WriteLine($"{(int)o} = {o}");

        // zamiana tekstu na wartość enum
        if (Enum.TryParse("Piatek", out DzienTygodnia wynik))
            Console.WriteLine($"Rozpoznano: {wynik}");
    }
}
```

3.2. Struktura struct

Struktura grupuje kilka powiązanych wartości (pól) i operacji na nich (metod) w jeden typ. Struktury są **typami wartościowymi** – przypisanie `c = b` kopiuje całą zawartość, a zmiana kopii nie wpływa na oryginał. Strukturę stosujemy dla niewielkich obiektów reprezentujących pojedynczą wartość (punkt, data, temperatura). Szczegółowe porównanie struktur i klas będzie tematem ćwiczenia 4.

Listing 1.11. Struktura Punkt z konstruktorem i metodą

```
using System;

struct Punkt
{
    public double X;
    public double Y;

    public Punkt(double x, double y)    // konstruktor struktury
    {
        X = x;
        Y = y;
    }
}
```

```

    public double OdlegloscOd(Punkt p)    // metoda struktury
    {
        double dx = X - p.X, dy = Y - p.Y;
        return Math.Sqrt(dx * dx + dy * dy);
    }
}

class Program
{
    static void Main()
    {
        Punkt a = new Punkt(0, 0);
        Punkt b = new Punkt(3, 4);
        Console.WriteLine($"Odległość A-B = {a.OdlegloscOd(b)}");    // 5

        Punkt c = b;          // struktura jest typem WARTOŚCIOWYM – kopiowana jest cała zawartość
        c.X = 100;
        Console.WriteLine($"b.X = {b.X}, c.X = {c.X}");              // b.X = 3, c.X = 100
    }
}

```

3.3. Krotki (tuple)

Krotka pozwala połączyć kilka wartości różnych typów bez definiowania nowego typu. Elementy mogą mieć nazwy (`student.Nazwisko`), a krotkę można **zdekonstruować** do osobnych zmiennych. Krotki są szczególnie przydatne do zwracania kilku wyników z metody (ćwiczenie 3).

Listing 1.12. Tworzenie, dekonstrukcja i zamiana wartości krotką (fragment – umieść wewnątrz metody Main)

```

// krotka (tuple) – szybkie grupowanie kilku wartości bez definiowania nowego typu
(string Nazwisko, int Rok, double Srednia) student = ("Kowalski", 2006, 4.25);
Console.WriteLine($"{{student.Nazwisko}}, ur. {{student.Rok}}, średnia {{student.Srednia}}");

var wymiary = (Szer: 1920, Wys: 1080);
Console.WriteLine($"Piksele: {{wymiary.Szer * wymiary.Wys}}");

// dekonstrukcja krotki do osobnych zmiennych
var (szer, wys) = wymiary;
Console.WriteLine($"Proporcje: {{(double)szer / wys:F3}}");

// zamiana wartości dwóch zmiennych bez zmiennej pomocniczej
int x = 1, y = 2;
(x, y) = (y, x);
Console.WriteLine($"x = {{x}}, y = {{y}}");

```

Konstrukcja	Kiedy stosować	Przykład
<code>enum</code>	zamknięty zbiór nazwanych wariantów	dni tygodnia, statusy zamówienia, poziomy trudności
<code>struct</code>	mały typ „wartość” z polami i metodami, używany wielokrotnie	punkt, wektor, temperatura, ułamek
krotka (,)	szybkie, lokalne zgrupowanie kilku wartości	wynik metody: (min, max, średnia)
<code>class</code> (ćw. 4)	obiekty z tożsamością, stanem i zachowaniem	student, konto bankowe, książka

3.4. Instrukcja if...else

Instrukcja `if` wykonuje blok kodu tylko wtedy, gdy warunek (wyrażenie typu `bool`) jest prawdziwy. Gałąź `else` wykonuje się w przeciwnym przypadku. Kaskada `else if` sprawdza warunki po kolei i wykonuje **tylko pierwszą** gałąź, której warunek jest prawdziwy – kolejność warunków ma więc znaczenie.

Listing 1.13. Kaskada `if...else if` – przeliczenie punktów na ocenę (fragment – umieść wewnątrz metody `Main`)

```
Console.WriteLine("Podaj liczbę punktów (0-100): ");
int pkt = int.Parse(Console.ReadLine());

if (pkt < 0 || pkt > 100)
{
    Console.WriteLine("Błędna liczba punktów!");
}
else if (pkt >= 91)
{
    Console.WriteLine("Ocena: 5,0");
}
else if (pkt >= 81)
{
    Console.WriteLine("Ocena: 4,5");
}
else if (pkt >= 71)
{
    Console.WriteLine("Ocena: 4,0");
}
else if (pkt >= 61)
{
    Console.WriteLine("Ocena: 3,5");
}
else if (pkt >= 51)
{
    Console.WriteLine("Ocena: 3,0");
}
else
{
    Console.WriteLine("Ocena: 2,0");
}
```

Dobre praktyki

- Zawsze używaj nawiasów klamrowych, nawet dla jednej instrukcji – zapobiega to błędom przy późniejszym dopisywaniu kodu.
- Nie stawiaj średnika po nawiasie warunku: `if (x > 3);` to pusta instrukcja warunkowa, a następny blok wykona się zawsze.
- Porównanie to `==`, a przypisanie to `=`. W C# zapis `if (x = 5)` jest błędem kompilacji – to dobra ochrona.
- Najpierw obsłuż przypadki błędne (walidacja), potem właściwą logikę – unikasz głębokiego zagnieżdżenia.

3.5. Instrukcja switch, wyrażenie switch i operator ?:

Instrukcja `switch` wybiera jeden z wielu wariantów na podstawie wartości wyrażenia (liczby, znaku, łańcucha, wartości `enum`). Każda sekcja `case` musi kończyć się instrukcją `break` (lub `return`) – w C# nie ma „przechodzenia” do następnej sekcji. Kilka etykiet może wskazywać ten sam blok. Sekcja `default` obsługuje wszystkie pozostałe przypadki.

Listing 1.14. Instrukcja switch – prosty kalkulator (fragment – umieść wewnątrz metody Main)

```

Console.WriteLine("Podaj działanie (+, -, *, /): ");
string op = Console.ReadLine();
double a = 12, b = 4, wynik;

switch (op)
{
    case "+":
        wynik = a + b;
        break;
    case "-":
        wynik = a - b;
        break;
    case "*":
    case "x":                // kilka etykiet dla jednego bloku
        wynik = a * b;
        break;
    case "/":
        wynik = a / b;
        break;
    default:
        Console.WriteLine("Nieznane działanie");
        wynik = double.NaN;
        break;              // w C# break jest obowiązkowy także w default
}
Console.WriteLine($"Wynik: {wynik}");

```

Od wersji C# 8 dostępne jest **wyrażenie switch**, które zwraca wartość i pozwala stosować wzorce (np. relacyjne `>=`, logiczne `or`, `and`, `not`). Operator warunkowy **warunek ? wartość1 : wartość2** jest zwięzłą formą wyboru jednej z dwóch wartości.

Listing 1.15. Wyrażenie switch ze wzorcami oraz operator warunkowy (fragment – umieść wewnątrz metody Main)

```

int pkt = 76;

// wyrażenie switch (C# 8+) ze wzorcami relacyjnymi (C# 9+)
string ocena = pkt switch
{
    < 0 or > 100 => "błąd",
    >= 91 => "5,0",
    >= 81 => "4,5",
    >= 71 => "4,0",
    >= 61 => "3,5",
    >= 51 => "3,0",
    _ => "2,0"                // _ oznacza „każdy inny przypadek”
};
Console.WriteLine($"Ocena: {ocena}");

// operator warunkowy ?: – krótki odpowiednik if/else zwracający wartość
int temp = -3;
string stan = temp < 0 ? "lód" : "woda";
Console.WriteLine(stan);

```


Polecenia – część 3

Polecenie 10. Równanie kwadratowe – wersja kompletna

poziom podstawowy · orientacyjny czas: 15 min

Napisz program rozwiązujący równanie $ax^2 + bx + c = 0$. Współczynniki wczytaj za pomocą `TryParse`. Program musi obsługiwać wszystkie przypadki:

- $a = 0$ – równanie liniowe $bx + c = 0$ (w tym: brak rozwiązań, nieskończenie wiele rozwiązań),
- $\Delta > 0$ – dwa pierwiastki, $\Delta = 0$ – jeden pierwiastek podwójny, $\Delta < 0$ – brak pierwiastków rzeczywistych (dla chętnych: wyświetl pierwiastki zespolone w postaci $p \pm qi$).

Wyniki wyświetl z dokładnością do 3 miejsc po przecinku. Porównując Δ z zerem, zastanów się nad problemem precyzji liczb `double`.

Punkt kontrolny – zgłoś prowadzącemu zakończenie realizacji tego polecenia.

Polecenie 11. Dni tygodnia i typ wyliczeniowy

poziom podstawowy · orientacyjny czas: 8 min

Zdefiniuj `enum DzieńTygodnia`. Program wczytuje numer dnia (1–7), zamienia go na wartość typu wyliczeniowego i za pomocą **wyrażenia switch** wyświetla informację: „dzień roboczy”, „sobota – zajęcia niestacjonarne” lub „niedziela – zajęcia niestacjonarne”. Dla numeru spoza zakresu wyświetl komunikat o błędzie (użyj `Enum.IsDefined`).

Polecenie 12. Rok przestępny i liczba dni miesiąca

poziom podstawowy · orientacyjny czas: 8 min

Program wczytuje rok i numer miesiąca, a następnie wyświetla liczbę dni w tym miesiącu. Rok jest przestępny, gdy jest podzielny przez 4 i niepodzielny przez 100 albo podzielny przez 400. Zapisz ten warunek jednym wyrażeniem logicznym. Wynik porównaj z wartością zwracaną przez `DateTime.DaysInMonth(rok, miesiac)`.

Polecenie 13. Struktura Punkt i ćwiartki układu

poziom rozszerzony (*) · orientacyjny czas: 15 min

Rozbuduj strukturę `Punkt` z listingu 1.11 o metodę `string Cwiartka()` zwracającą numer ćwiartki układu współrzędnych („I”, „II”, „III”, „IV”) lub informację, że punkt leży na osi albo w początku układu. Wczytaj współrzędne trzech punktów A, B, C, wyświetl ich ćwiartki, długości boków trójkąta ABC oraz informację, czy z punktów da się zbudować trójkąt (nierówność trójkąta) i czy jest on prostokątny (twierdzenie Pitagorasa z tolerancją).

Polecenie 14. Taryfa za przejazd – krotki i warunki złożone

poziom rozszerzony (*) · orientacyjny czas: 10 min

Cena biletu zależy od wieku i statusu: dzieci do 7 lat – 0 zł, uczniowie i studenci do 26 lat – 49% zniżki, seniorzy od 70 lat – 0 zł, pozostali – cena pełna 4,40 zł. Dane pasażera przechowaj w krotce (`int Wiek`, `bool Student`), a cenę wyznacz jednym wyrażeniem switch ze wzorcami (wskazówka: wzorzec krotki (`< 7, _`) => `0m`). Kwotę wyświetl w formacie walutowym.

Punkt kontrolny – zgłoś prowadzącemu zakończenie realizacji tego polecenia.

Część 4. Pętle

4.1. Rodzaje pętli

Pętla powtarza blok instrukcji (ciało pętli) tak długo, jak spełniony jest warunek. Każdy obrót nazywamy **iteracją**. Wybór pętli zależy od tego, czy liczba powtórzeń jest znana z góry i czy ciało ma wykonać się co najmniej raz.

Pętla	Warunek sprawdzany	Minimalna liczba wykonań	Typowe zastosowanie
<code>for (init; warunek; krok)</code>	przed każdą iteracją	0	znana liczba powtórzeń, przechodzenie po indeksach tablicy
<code>while (warunek)</code>	przed każdą iteracją	0	powtarzanie „dopóki” – liczba powtórzeń nieznana
<code>do { } while (warunek);</code>	po każdej iteracji	1	wczytywanie danych do skutku, menu programu
<code>foreach (var x in kolekcja)</code>	czy są kolejne elementy	0	odczyt wszystkich elementów tablicy, listy, łańcucha

Listing 1.16. Cztery rodzaje pętli (fragment – umieść wewnątrz metody Main)

```
// for - gdy znamy liczbę powtórzeń
for (int i = 1; i <= 5; i++)
    Console.WriteLine($"{i * i} ");
Console.WriteLine();

// while - warunek sprawdzany PRZED każdym obrotem (może nie wykonać się ani razu)
int n = 1000, kroki = 0;
while (n > 1)
{
    n /= 2;
    kroki++;
}
Console.WriteLine($"1000 można podzielić przez 2 aż {kroki} razy");

// do...while - warunek sprawdzany PO obrocie (wykona się co najmniej raz)
int liczba;
do
{
    Console.WriteLine("Podaj liczbę dodatnią: ");
} while (!int.TryParse(Console.ReadLine(), out liczba) || liczba <= 0);
Console.WriteLine($"Dziękuję, wczytano {liczba}");

// foreach - przejście po wszystkich elementach kolekcji
foreach (char znak in "C#")
    Console.WriteLine($"Znak '{znak}' ma kod {(int)znak}");
```

Pętla **for** składa się z trzech części: **inicjalizacji** (wykonywanej raz), **warunku** (sprawdzanego przed każdą iteracją) i **kroku** (wykonywanego po każdej iteracji). Zmienna sterująca zadeklarowana w nagłówku istnieje tylko wewnątrz pętli. Pętla nieskończona to `while (true) { ... }` lub `for (;;) { ... }` – musi zawierać instrukcję **break** albo **return**.

4.2. Instrukcje break i continue

Listing 1.17. Pominięcie iteracji i przerwanie pętli (fragment – umieść wewnątrz metody Main)

```
// suma liczb nieparzystych od 1 do 99, przerywana gdy suma przekroczy 1000
int suma = 0;
for (int i = 1; i < 100; i++)
{
    if (i % 2 == 0)
        continue;           // pominięcie reszty obrotu – przejdź do następnego i
    suma += i;
    if (suma > 1000)
    {
        Console.WriteLine($"Przekroczono 1000 przy i = {i}");
        break;               // natychmiastowe wyjście z pętli
    }
}
Console.WriteLine($"Suma = {suma}");
```

4.3. Pętle zagnieżdżone

Pętla wewnątrz innej pętli wykonuje się w całości dla każdej iteracji pętli zewnętrznej – liczba wykonań ciała wewnętrznego to iloczyn liczby iteracji obu pętli (dla tabliczki mnożenia $9 \cdot 9 = 81$). Pętle zagnieżdżone są podstawą przetwarzania tablic dwuwymiarowych w ćwiczeniu 2.

Listing 1.18. Tabliczka mnożenia z wyrównaniem kolumn (fragment – umieść wewnątrz metody Main)

```
// tabliczka mnożenia 1..9 – pętla zagnieżdżona
Console.Write(" ");
for (int k = 1; k <= 9; k++) Console.Write($"{k,4}");
Console.WriteLine();
for (int w = 1; w <= 9; w++)
{
    Console.Write($"{w,4}");
    for (int k = 1; k <= 9; k++)
    {
        Console.Write($"{w * k,4}");
    }
    Console.WriteLine();           // koniec wiersza
}
```

4.4. Szkielet programu z menu

Program konsolowy, który ma obsługiwać wiele poleceń użytkownika, budujemy według schematu: **pętla główna** → **wyświetlenie menu** → **wczytanie wyboru** → **switch** → **wykonanie akcji**. Zakończenie następuje po wybraniu opcji „koniec”, która zmienia wartość zmiennej sterującej pętlą. Ten schemat będzie podstawą projektu końcowego.

Listing 1.19. Szkielet programu z menu tekstowym

```
using System;

class Program
{
    static void Main()
    {
        bool dziala = true;
        while (dziala)           // główna pętla programu konsolowego
        {
            Console.WriteLine();
            Console.WriteLine("=== KALKULATOR JEDNOSTEK ===");
            Console.WriteLine("1 - km/h -> m/s");
            Console.WriteLine("2 - °C -> °F");
        }
    }
}
```

```

Console.WriteLine("0 - koniec");
Console.Write("Wybór: ");
string wybor = Console.ReadLine();

switch (wybor)
{
    case "1":
        Console.Write("Prędkość [km/h]: ");
        double v = double.Parse(Console.ReadLine());
        Console.WriteLine($"= {v / 3.6:F2} m/s");
        break;
    case "2":
        Console.Write("Temperatura [°C]: ");
        double c = double.Parse(Console.ReadLine());
        Console.WriteLine($"= {c * 9 / 5 + 32:F1} °F");
        break;
    case "0":
        dziala = false;
        break;
    default:
        Console.ForegroundColor = ConsoleColor.Red;
        Console.WriteLine("Nieznana opcja!");
        Console.ResetColor();
        break;
}
}
Console.WriteLine("Do widzenia.");
}
}

```

Jak uniknąć błędów w pętlach

- **Pętla nieskończona** – sprawdź, czy w ciele pętli zmienia się wartość wpływająca na warunek.
- **Błąd o jeden** (off-by-one) – zastanów się, czy granica ma być $<$ czy $\leq$; prześledź pierwszą i ostatnią iterację na kartce lub w debugerze.
- **Nie modyfikuj zmiennej sterującej pętli for** wewnątrz jej ciała – utrudnia to analizę.
- W debugerze możesz ustawić **warunkowy punkt przerywania** (prawy przycisk na czerwonej kropce → Warunki), np. `i == 57`, aby zatrzymać program w konkretnej iteracji.

Polecenia – część 4

Polecenie 15. Wartości funkcji trzema pętlami

poziom podstawowy · orientacyjny czas: 10 min

Wyświetl w formie tabeli (kolumny x i y) wartości funkcji $y = x^2 - 4x + 3$ dla x od -2 do 6 z krokiem $0,5$.

Wykonaj zadanie trzykrotnie: pętlą `for`, `while` i `do...while`. Która pętla jest najbardziej naturalna w tym zadaniu?

Zwróć uwagę na problem kumulacji błęd przy sumowaniu kroku $0,5$ – zaproponuj wersję, w której zmienną sterującą jest licznik całkowity.

Polecenie 16. Gra „Zgadnij liczbę” – wersja rozbudowana

poziom podstawowy · orientacyjny czas: 15 min

Komputer losuje liczbę z przedziału 1–100 (`Random.Shared.Next(1, 101)`). Użytkownik ma 7 prób. Po każdej nieudanej próbie program podpowiada „za mało” / „za dużo” i podaje liczbę pozostałych prób. Niepoprawne dane (nie-liczba, liczba spoza zakresu) nie zużywają próby. Po zakończeniu gry program pyta, czy zagrać ponownie (t/n).

Pytanie analityczne: dlaczego przy optymalnej strategii 7 prób zawsze wystarcza? (wskazówka: $2^7 = 128$).

Punkt kontrolny – zgłoś prowadzącemu zakończenie realizacji tego polecenia.

Polecenie 17. Statystyka wczytywanych liczb

poziom podstawowy · orientacyjny czas: 10 min

Program wczytuje kolejne liczby całkowite, aż użytkownik wpisze 0 (zero nie jest wliczane). Następnie wyświetla: liczbę wczytanych wartości, sumę, średnią, wartość najmniejszą i największą oraz liczbę wartości parzystych. Nie używaj tablic – wszystkie statystyki aktualizuj „na bieżąco” w pętli. Obsłuż przypadek, gdy pierwszą liczbą jest 0.

Polecenie 18. Wzory z gwiazdek

poziom podstawowy · orientacyjny czas: 12 min

Wczytaj wysokość h i za pomocą pętli zagnieżdżonych narysuj: (a) trójkąt prostokątny, (b) piramidę wyśrodkowaną, (c) pusty kwadrat (tylko obwód), (d) literę X w kwadracie $h \times h$.

Przykładowy przebieg programu:

$h = 4$ (piramida)

```
*
***
*****
*****
```

Polecenie 19. Liczby doskonałe i NWD

poziom rozszerzony (*) · orientacyjny czas: 12 min

Wyświetl wszystkie liczby doskonałe mniejsze od 10 000 (liczba doskonała jest równa sumie swoich dzielników właściwych, np. $6 = 1 + 2 + 3$). Policz, ile iteracji pętli wewnętrznej wykonał program, i zaproponuj modyfikację, która zmniejszy tę liczbę (wystarczy sprawdzać dzielniki do $\sqrt{n}$).

Wczytaj dwie liczby naturalne i wyznacz ich największy wspólny dzielnik algorytmem Euklidesa (wersja z resztą z dzielenia) w pętli `while`.

Polecenie 20. Kalkulator z menu

poziom rozszerzony (*) · orientacyjny czas: 15 min

Na podstawie listingu 1.19 napisz kalkulator z menu: 1 – suma, 2 – różnica, 3 – iloczyn, 4 – iloraz, 5 – potęga, 6 – pierwiastek, 0 – koniec. Wszystkie dane wczytuj z walidacją (pętla `do...while` z `TryParse`). Obsłuż dzielenie przez zero i pierwiastek z liczby ujemnej. Po każdej operacji program czeka na naciśnięcie klawisza i czyści ekran (`Console.Clear()`).

Punkt kontrolny – zgłoś prowadzącemu zakończenie realizacji tego polecenia.

Polecenie 21. Ciąg Collatza

poziom zaawansowany (**) · orientacyjny czas: praca własna

Dla liczby n ciąg Collatza powstaje według reguły: jeśli n jest parzyste, następny wyraz to $n/2$, w przeciwnym razie $3n + 1$; ciąg kończy się po osiągnięciu 1. Znajdź liczbę z przedziału 1–1 000 000, dla której ciąg jest najdłuższy. Użyj typu `long` dla wyrazów ciągu (dlaczego?) i zmierz czas obliczeń klasą `System.Diagnostics.Stopwatch`.

Podsumowanie i praca własna

Pytania kontrolne

1. Czym różni się kod pośredni IL od kodu maszynowego i jaką rolę pełni kompilator JIT?
2. Dlaczego wyrażenie `7 / 2` ma wartość 3, a `7 / 2.0` wartość 3,5?
3. Kiedy należy użyć typu `decimal` zamiast `double`? Podaj przykład z praktyki.
4. Jaka jest różnica między `int.Parse` a `int.TryParse`? Która metoda jest właściwa do wczytywania danych od użytkownika i dlaczego?
5. Co się stanie, gdy w instrukcji `switch` pominiemy `break`?
6. Czym różni się pętla `while` od `do...while`? Podaj zadanie, w którym tylko jedna z nich jest naturalnym wyborem.
7. Co oznacza, że struktura jest typem wartościowym? Jak to sprawdzić w programie?
8. Ile razy wykona się ciało pętli `for (int i = 10; i > 0; i -= 3)`? Jakie wartości przyjmie `i`?

Praca własna przed ćwiczeniem 2

- Dokończ wszystkie polecenia podstawowe oraz co najmniej dwa polecenia rozszerzone.
- Przeczytaj w książce M. Lisa lekcje 12 i 13 (tablice jednowymiarowe i wielowymiarowe).
- Zastanów się nad tematem projektu indywidualnego (lista propozycji znajduje się w ćwiczeniu 4) – już teraz możesz przygotować szkielet programu z menu na podstawie listingu 1.19.

Kryteria zaliczenia ćwiczenia

Wymaganie	Sposób weryfikacji
Wykonanie poleceń podstawowych oznaczonych jako punkty kontrolne (4, 9, 10, 14, 16, 20)	prezentacja działającego programu prowadzącemu w trakcie zajęć
Poprawna walidacja danych wejściowych (<code>TryParse</code>)	test programu na danych błędnych
Czytelność kodu: nazwy, wcięcia, komentarze	przegląd kodu
Polecenia rozszerzone i zaawansowane	dodatkowe punkty aktywności